

Raushan Kumar

DATA MODELING

- ✓ Types of Data Models
- ✓ Key Data Modeling Concepts
- ✓ Data Modeling Techniques
- ✓ Advanced Concepts in Data Modeling
- ✓ Real-Time Data Modeling Considerations

Raushan Kumar

<https://www.linkedin.com/in/raushan-kumar-553154297/>

DATA MODELING

Data modeling is the process of creating a visual representation of a data system, which helps in structuring, organizing, and managing data for efficient use. It involves defining the relationships, attributes, and the flow of data. A good data model helps optimize performance, ensures data integrity, and facilitates easy retrieval and maintenance.

Here's a detailed look at the key concepts of data modelling with real-time examples:

1. Types of Data Models

There are three primary types of data models: **Conceptual**, **Logical**, and **Physical**. These models are built in sequence, where each model refines and adds detail to the previous one.

a. Conceptual Data Model

- **Purpose:** High-level, abstract representation of the business needs and data requirements.
- **Focus:** Entities, relationships, and high-level business rules.
- **Example:** In an e-commerce platform, the conceptual model may highlight entities like *Customer*, *Product*, *Order*, and *Payment* without worrying about how the data will be stored.

b. Logical Data Model

- **Purpose:** More detailed than the conceptual model, but still independent of physical aspects like technology.
- **Focus:** Defines entities, attributes, and relationships, along with rules and constraints.
- **Example:** In an e-commerce context, the logical model specifies that the *Customer* entity has attributes like *Customer_ID*, *Name*, *Email*, and relationships like *Customer* places an *Order*.

c. Physical Data Model

- **Purpose:** Defines how the data will be stored in the database, with a focus on performance and optimization.
 - **Focus:** Tables, columns, data types, indexes, and keys.
 - **Example:** For the e-commerce store, the physical model would specify the structure of tables like *Customer*, *Product*, *Order*, and how they are indexed for faster queries (e.g., an index on *Customer_ID* for quick lookups).
-

2. Key Data Modeling Concepts

a. Entities and Attributes

- **Entity:** Represents a real-world object, concept, or event.
- **Attribute:** A property or characteristic of an entity.
- **Example:** In a banking system, *Customer* is an entity with attributes like *Customer_ID*, *Name*, and *Address*.

b. Relationships

- **Relationship:** Represents how entities are related to each other.
- **Types of Relationships:**
 - **One-to-One (1:1):** One entity relates to only one other entity.
 - **Example:** A *Person* has one *Passport*.
 - **One-to-Many (1:N):** One entity relates to many instances of another entity.
 - **Example:** A *Department* has many *Employees*.
 - **Many-to-Many (M:N):** Many entities relate to many others.

- **Example:** *Students* enroll in many *Courses*.

c. Keys

- **Primary Key:** A unique identifier for records in a table.
- **Foreign Key:** A reference to the primary key in another table, establishing a relationship between two tables.
- **Example:** In a sales system, *Order_ID* could be the primary key of the *Orders* table, and *Customer_ID* in the *Orders* table could be a foreign key referencing the *Customer* table.

d. Normalization

- **Normalization:** The process of organizing data to minimize redundancy and dependency.
- **Levels of Normalization:**
 - **1st Normal Form (1NF):** Ensures that there are no repeating groups or arrays in a table.
 - **2nd Normal Form (2NF):** Eliminates partial dependency (non-key attributes depend entirely on the primary key).
 - **3rd Normal Form (3NF):** Removes transitive dependency (non-key attributes should not depend on other non-key attributes).
 - **Boyce-Codd Normal Form (BCNF):** A stricter version of 3NF.
- **Example:** In an e-commerce system, a table with *Customer Name*, *Customer Address*, and *Order ID* might need to be split into separate tables to eliminate redundancy.

e. Denormalization

- **Denormalization:** The process of combining tables to improve query performance, especially for read-heavy systems.
 - **Example:** In a reporting system, combining multiple normalized tables (e.g., *Orders*, *Customers*, and *Products*) into a single table to reduce the number of joins in complex queries.
-

3. Data Modeling Techniques

a. Entity-Relationship Diagram (ERD)

- **Purpose:** Visual representation of the data model showing entities, attributes, and relationships.
- **Example:** An ERD of a university database might show entities like *Student*, *Course*, *Professor*, and relationships like *Student enrolls in Course*.

b. Star Schema and Snowflake Schema (Used in Data Warehousing)

- **Star Schema:** A central fact table is connected to dimension tables, often used in OLAP (Online Analytical Processing) systems.
 - **Example:** In a sales data warehouse, the central fact table might be *Sales_Fact*, connected to dimension tables like *Time*, *Product*, and *Customer*.
- **Snowflake Schema:** A more normalized version of the star schema where dimension tables are split into multiple related tables.
 - **Example:** In the same sales data warehouse, the *Product* dimension might be split into *Product_Category*, *Product_Supplier*, and *Product_Details*.

c. Dimensional Modeling

- **Purpose:** Focuses on designing databases for analytical purposes, typically in data warehouses.
 - **Fact Table:** Contains quantitative data for analysis.
 - **Dimension Table:** Contains descriptive data (attributes) related to the facts.
 - **Example:** In a sales system, the *Sales_Fact* table might include *Revenue*, while the *Product* dimension includes attributes like *Product_Name*, *Category*, and *Brand*.
-

4. Advanced Concepts in Data Modeling

a. Slowly Changing Dimensions (SCD)

- **Purpose:** Deals with how to manage changes in data over time, especially in data warehouses.
- **Types:**
 - **Type 1:** Overwrites old data with new data (no history).
 - **Type 2:** Creates new records to maintain historical data.
 - **Type 3:** Keeps both the old and new values in the same record.
- **Example:** In a customer database, if a customer's address changes, using SCD Type 2 would create a new record to track the change, maintaining both the old and new addresses.

b. Data Vault Modeling

- **Purpose:** A flexible and scalable modeling approach for large and complex data warehouses, focused on auditability and traceability.
 - **Components:**
 - **Hubs:** Keys of business entities.
 - **Links:** Relationships between hubs.
 - **Satellites:** Historical data related to the hubs and links.
 - **Example:** In a financial system, hubs might represent *Customer* and *Account*, links might represent *Transactions*, and satellites store historical changes to customer details.
-

5. Real-Time Data Modeling Considerations

a. Event-Driven Architectures

- Data models for real-time systems (e.g., IoT platforms, financial trading systems) need to accommodate event-based data.
- **Example:** In a stock trading platform, an event-driven data model might track trades in real time, storing each event in an event log.

b. NoSQL Data Models

- When dealing with unstructured or semi-structured data, NoSQL databases like MongoDB, Cassandra, and DynamoDB are used, with flexible data models like document, key-value, and column-family.
- **Example:** A social media platform may store user posts as documents in MongoDB, where each post contains varying attributes like text, images, and metadata.

c. Data Lakes and Schema-on-Read

- In modern data architecture, data lakes allow for storing raw, unprocessed data. The schema is applied when the data is read, not when it is written.
 - **Example:** In an e-commerce company, data from various sources (e.g., transaction logs, customer activity, external data feeds) is stored in a data lake, and the structure is defined when data is queried for analysis.
-

Conclusion

Data modeling is an essential part of designing systems that manage, store, and analyze data. It involves various techniques and methodologies, from ER diagrams to star schemas and even modern approaches like data vault modeling. The key is understanding your business requirements and selecting the right modeling techniques to ensure that the system is efficient, scalable, and can meet the demands of real-time and historical data analysis.

By: Raushan Kumar

Please follow for more such content:

<https://www.linkedin.com/in/raushan-kumar-553154297/>